
Implementation of Fast and Compact Vision Transformer From Scratch

Penghao Zhang*, Xiwen Chen*, Zhihan Xu*, Wenyang Yao*
University of Southern California
{penghaoz,xiwenc,zhihanxu}@usc.edu

Abstract

Vision Transformer (ViT) is gaining popularity in the realm of computer vision, owing to their unique architecture and the utilization of self-attention mechanisms. In this project, we implement a compact version of ViT (c-ViT) for fast object recognition. The implementation is from scratch to gain a deeper understanding of the model's architecture, functionality, and performance. The c-ViT is set as our baseline model. We evaluate the baseline on CIFAR-10 and CIFAR-100 and obtain 76.79% and 45.41% test accuracies respectively. Afterward, we make two modifications to the baseline model: 1. We remove the class token embedding; 2. We adopt the sequence pooling method to extract information from all output tokens post transformer encoder rather than relying on a single class token. We denote the modified model as modified compact ViT (mc-ViT). We further implement a fast multi-head attention module where all heads are processed simultaneously with the merged query, key, and value. We denote the fast versions of our two models as f-c-ViT, and f-mc-ViT, respectively. The experiment results show that the mc-ViT can improve the accuracy by 1.4% on average (w. & w/o. fast version) compared with c-ViT on CIFAR-100, with only marginal (0.05%) model size increase. For the training speed, both f-c-ViT and f-mc-ViT can speed up the training process compared against c/mc-ViT. Our code is available at <https://github.com/francischen0724/Image-classification-by-CVT-based-on-Vit-from-scratch>.

1 Introduction

The field of computer vision has witnessed significant advancements with the advent of deep learning, particularly Convolutional Neural Networks (CNNs) [1]. While CNNs have been the dominant architecture for image classification, object detection, and segmentation tasks, recent innovations have demonstrated the potential of transformer-based models [2] to outperform traditional methods. Vision Transformer (ViT) [3] leverages the self-attention mechanism, a concept popularized by transformer architectures in natural language processing, to analyze image data without convolutional layers.

This project explores the implementation of a ViT from scratch, focusing on understanding its key components, including patch embedding, positional encoding, and multi-head self-attention. By implementing the model manually, the goal is to grasp the theoretical underpinnings and practical considerations of transformer architectures in vision tasks. Therefore, we implement a compact version of ViT (c-ViT) with only 4 hidden layers and 4 attention heads, which is our baseline model. Then, we modify c-ViT model by eliminating the requirement of class token and using the sequence pooling mechanism post encoder to better utilize the information. The modified model is denoted as mc-ViT. For fair comparison, mc-ViT adopts the same configurations as c-ViT, such as number of

*equal contribution

hidden layers, hidden size, and number of attention heads, etc. We evaluate both c-ViT and mc-ViT on the CIFAR-10 [4] and CIFAR-100 [5] datasets.

The experimental results show that mc-ViT can achieve 1.78% larger accuracy than c-ViT on CIFAR-100 [5]. The results prove the observation that the sequence pooling method can extract better information compared with the traditional class tokenization method.

To further speed up the model runtime speed, We implement a fast multi-head attention module where all heads are processed simultaneously with a merged query, key, and value. Our models adopting the fast attention module are denoted as f-c-ViT, and f-mc-ViT, respectively. The experiment results show that both f-c-ViT and f-mc-ViT can speed up the training process.

In summary, our key findings are as follows.

- mc-ViT can achieve larger accuracies than c-ViT (our baseline) on both CIFAR-10 and CIFAR-100 datasets, e.g., 1.78% on CIFAR100. However, the model size of mc-ViT is only 0.05% larger c-ViT.
- mc-ViT makes the model converge faster during the training process than c-ViT.
- Fast-mode models f-c/mc-ViT can reduce the training time compared with c/mc-ViT.

2 Methods

The model-related parameters are summarized in Table 1.

Table 1: Model-related Parameters

Param	Description
B	Batch size
L	Sequence length; # of patches per image
H	Hidden size; input embedding size
$\#head$	# of attention heads
h	attention head size, we set it as $H/\#head$
l	model layers

The model architectures of c-ViT and mc-ViT is shown in Fig. 1. It is worth noting that mc-ViT does not require adding the class token embedding. The sequence pooling, shown in Fig. 2, is added after encoder to extract information across all tokens rather than only relying on the single class token.

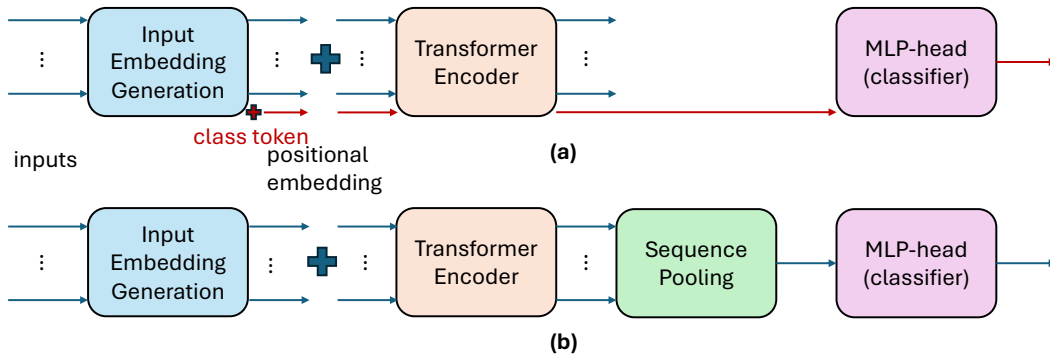


Figure 1: Model structure comparison: (a) c-ViT; (b) mc-ViT

For fast versions f-c-ViT and f-mc-ViT, they adopt the fast multi-head attention module. Fig. 3 shows the comparison between traditional multi-head attention module and the fast version. There are two levels of optimizations for fast version. On the one hand, we use a single layer to project for all (three) of them simultaneously rather than using a single layer for each of them. On the other hand, all heads are processed simultaneously. Both optimization increases the speed of model runtime due to the simultaneous process of all heads with merged query, key, and value projections.

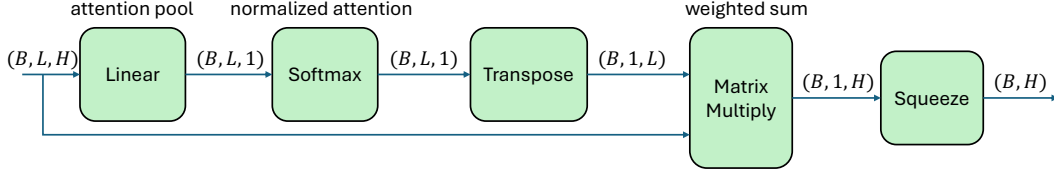


Figure 2: Sequence pooling structure

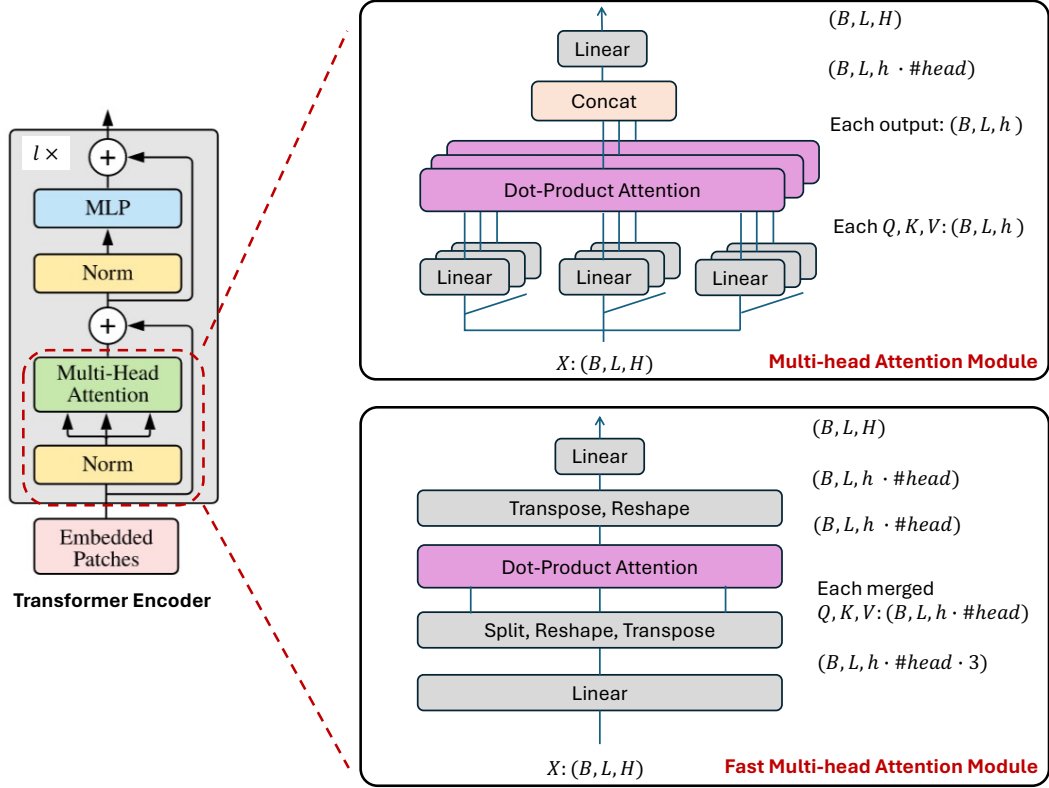


Figure 3: Comparison of multi-head attention modules within c/mc-ViT and f-c/mc-ViT

The model size comparison against the original ViT [3] is shown in Table 2. As mentioned before, c-ViT (our baseline) and mc-ViT adopt the same configurations for fair comparison. The mc-ViT has a marginally larger parameter size due to the added sequence pooling post encoding layer.

Table 2: Model Variant Comparison

Model	l	H	MLP size	#head	Params
c-ViT	4	48	192	4	92544*
mc-ViT	4	48	192	4	92592*
f-c-ViT	4	48	192	4	101760*
f-mc-ViT	4	48	192	4	101808*
Original-ViT [3]	12	768	3072	12	86M [†]

* Total model size including the final classifier for CIFAR-10 [4].

[†] The size does not include MLP head - the classifier.

We adopt the ADAM optimizer [6] for the training process of our models. The cross-entropy loss function is utilized. Details of implementation and experiment setup will be shown in the following Sec. 3 and Sec. 4.

3 Implementation

We manually implement our ViT models from scratch with three main functional components as follows.

- **Input embedding generation:** This includes patch cropping, patch flattening, patch embedding generation, class embedding initialization (*N.A.* for (f)-mc-ViT), and positional embedding generation. The final input embedding is obtained by concatenating the class embedding with flattened patch embedding and then summing with the positional embedding.
- **Transformer encoder:** This is the core implementation in this project, which mainly includes multi-head self-attention and multi-layer perceptron (MLP). Our transformer encoder has 4 layers. Each layer includes one Multi-Head Attention and one MLP, as shown in Fig. 3. The multi-head attention is implemented by instantiating 4 attention heads. Each attention head comprises three projection layers for query, key, and value, respectively. After projecting input The attention output is calculated based on the Eq. 1 below.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

The attention outputs from each attention head are concatenated together. After concatenation, a linear layer is applied to get the multi-head attention output.

For the MLP, we implement two dense layers with one GELU activation [7] between them. The input and output dimensions of MLP are the same, which is $B \times L \times H$. The intermediate size between two dense layers is $4 \times H$.

As mentioned, f-c-ViT and f-mc-ViT adopts the fast multi-head attention module shown in Fig. 3. The merged Q, K, V are computed simultaneously for all heads. The dimension of Q, K, V in Eq. 1 changes from (B, L, h) to $(B, L, h \cdot \#head)$. Therefore, we do not need the concatenation process.

- **MLP Head:** MLP-head is the final classifier that is a single linear layer to project the encoder's output to the number of classes. For CIFAR-10 [4] and CIFAR-100 [5] datasets we used in this project, number of classes are 10 and 100, respectively.

4 Experiments

4.1 Experiment Setup

The experiments are carried out on a single Nvidia GeForce RTX 4090 GPU paired with a 16-core AMD EPYC 9354 processor. Our implementation is based on Python 3.11.8 and Pytorch 2.2.2 [8] with CUDA ToolKit 12.1. The training process used a batch size of 256 and was conducted for 100 epochs. As mentioned, the ADAM optimizer [6] is used with an initial learning rate of 0.01 for all models. The cross-entropy loss function is adopted for classification.

The model-related parameter and training-related parameter, including hyperparameter configurations, are shown in Table 3.

Table 3: Parameter configurations

Parameter	B	L	H	h	l	$\#head$	# of epochs	learning rate
Value	256	64	48	12	4	4	100	0.01

The c/mc-ViT and f-c/mc-ViT, four models in total, are evaluated on CIFAR-10 [4] and CIFAR-100 [5] datasets, respectively. Since the two datasets have different numbers of classes, the MLP head, which is the final classifier, is modified accordingly.

4.2 Results and Analysis

Table 4 shows the performance of our models on both datasets in terms of both test accuracy and training runtime. It is clear that mc-ViT achieves 0.3% and 1.4% larger accuracies (averaged on

Table 4: Model performance and experiment results

Dataset	Model	Accuracy (%)	Training time (s)
CIFAR-10 [4]	c-ViT	76.79	1069.71
	mc-ViT	76.98	1048.73
	f-c-ViT	76.52	1058.21
	f-mc-ViT	76.88	1047.97
CIFAR-100 [5]	c-ViT [†]	45.41	1037.24
	mc-ViT	47.19	1045.52
	f-c-ViT	46.93	1035.85
	f-mc-ViT	47.96	1043.04

w. and w/o. fast mode) than c-ViT on CIFAR-10 and CIFAR-100 respectively. The accuracy improvement of the mc-ViT is more significant on CIFAR-100 than CIFAR-10. We acknowledge that the accuracies are not significant due to the compact model. In terms of the training speed, f-c-ViT is faster than c-ViT, and f-mc-ViT is faster than mc-ViT on both datasets.

Fig. 4 and Fig. 5 show the training processes of four models on CIFAR-10 and CIFAR-100, respectively. It is worth noting that mc-ViT can achieve faster converging speed than c-ViT on both datasets no matter with or without fast mode.

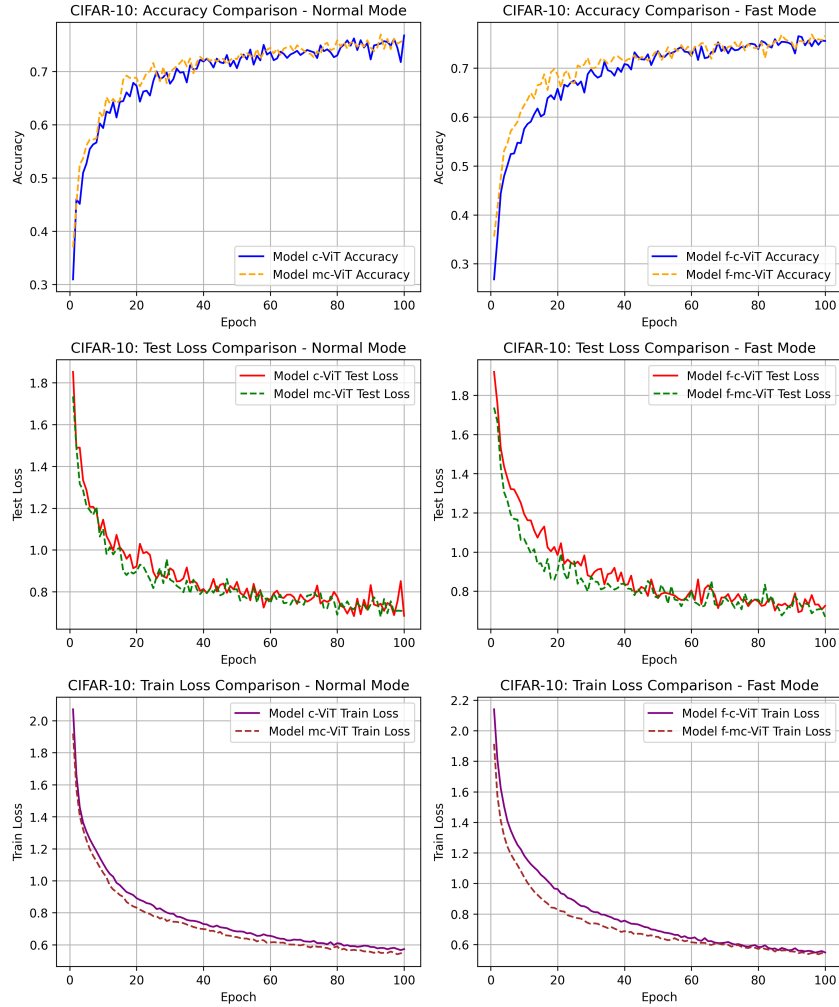


Figure 4: Comparison of training progress on CIFAR-10 datasets.

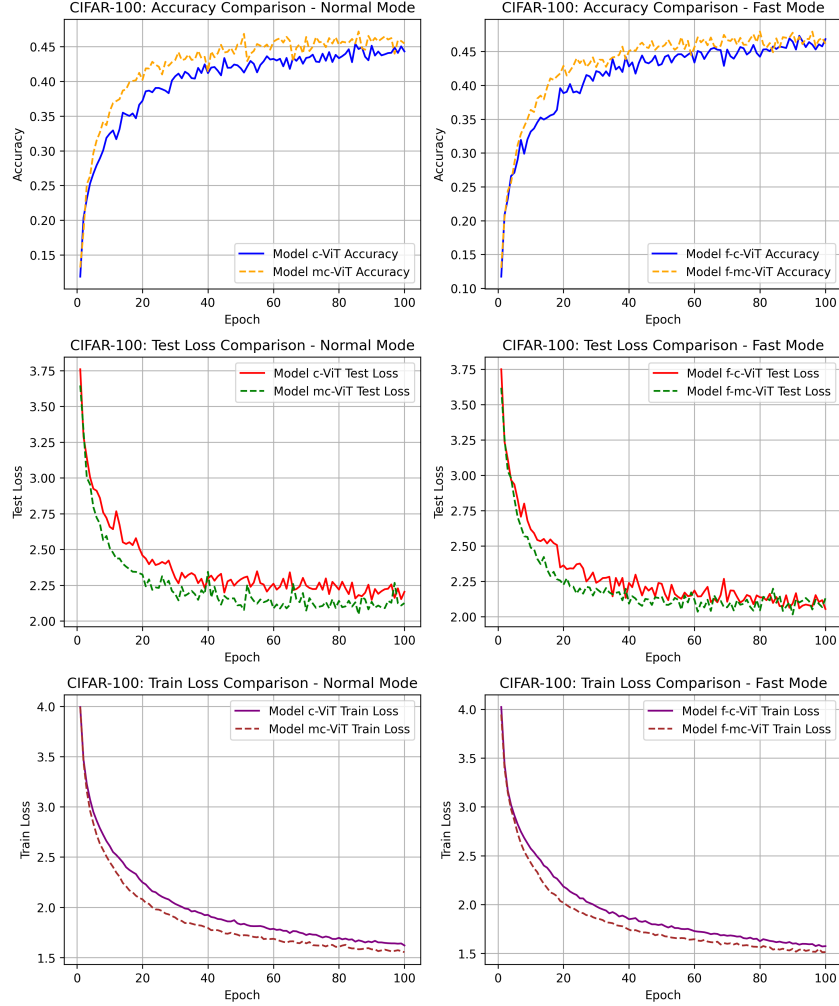


Figure 5: Comparison of training progress on CIFAR-100 datasets.

5 Conclusion

In this project, we implement a compact ViT (c-ViT) from scratch and a modified ViT (mc-ViT) by removing the class token and adding a sequence pooling layer post transformer encoder. Moreover, we implement fast versions of both models, denoted as f-c/mc-ViT, by introducing a fast multi-head attention module. The fast multi-head attention module can process all heads simultaneously with the merged query, value, and key. We evaluate four models (f-c/mc-ViT) on CIFAR-10 and CIFAR-100 datasets. The experiment results show that the mc-ViT can improve the accuracy by 0.3% and 1.4% on average (w. & w/o. fast mode) compared with c-ViT on CIFAR-10 and CIFAR-100, respectively. mc-ViT also achieves faster training convergence than c-ViT. As for the training speed, faster versions require less training time on both datasets. In the future, we plan to increase our model complexity to improve the accuracies and evaluate on larger datasets, e.g., ImageNet.

References

- [1] Yann LeCun, Koray Kavukcuoglu, and Clément Farabet. Convolutional networks and applications in vision. In *Proceedings of 2010 IEEE international symposium on circuits and systems*, pages 253–256. IEEE, 2010.
- [2] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

- [3] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [4] Alex Krizhevsky. Learning multiple layers of features from tiny images. *Technical Report*, 2009.
- [5] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-100 (canadian institute for advanced research). *Technical Report*, 2009.
- [6] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [7] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [8] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024.